Pokaz programu do 5 minut

Vladyslav Rosol i Olesia Voronina, grupa 22 INF-SP/B.

Na pokaz przeznaczamy 4 minuty i 45 sekund. Pozostałe 15 sekund zostawiamy na zmianę okna lub krótką przerwę. Pytania prowadzącego mogą wydłużyć spotkanie. Podział wypowiedzi między autorów jest propozycją do przećwiczenia.

Przed rozpoczęciem odmierzania czasu

1. Rozpakuj `symulatorPoS-do-oddania.zip` i otwórz folder `symulatorPoS`.

2. Otwórz `symulator_gui.exe`. Wskaż `przyklady/konfiguracja.txt` i `przyklady/zgloszenia.txt`. Nie uruchamiaj jeszcze przebiegu.

3. Otwórz PowerShell w tym samym folderze. Przygotuj polecenie z części o dużym zestawie.

4. Otwórz stronę z wynikami w `sprawozdanie.pdf`. Otwórz też `Symulator.cpp`, `Statystyki.cpp` i diagramy.

5. Ustaw czytelną wielkość tekstu. Wyłącz odliczanie przed rozpoczęciem pokazu, a następnie uruchom stoper.

0:00 do 0:25. Co robi program

Vladyslav:

"Nasz program symuluje obsługę transakcji przez ograniczoną liczbę walidatorów. Czas liczymy w krokach. Porównujemy FIFO, czyli kolejność nadejścia, i FEE, czyli pierwszeństwo wyższej opłaty. Stake jest wagą w losowaniu wolnego walidatora. Model obejmuje przydział pracy; nie tworzymy w nim bloków ani sieci blockchain."

0:25 do 1:15. Mały przykład w GUI

Olesia uruchamia przygotowany zestaw trzech transakcji. Pokazuje zakładkę FIFO, potem FEE.

"Każda transakcja trwa dwa kroki i przychodzi w czasie zero. Opłaty to 1, 9 i 4. FIFO daje kolejność 1, 2, 3, a FEE daje 2, 3, 1. Średnie oczekiwanie wynosi dwa kroki, maksymalne cztery. Wszystkie zadania kończą się w terminie."

Kliknij nagłówek Transakcja, a potem przycisk przywracania kolejności.

"Domyślnie tabela pokazuje zakończenia. Możemy uporządkować sam widok po ID lub czasie. Sortowanie nie zmienia wyniku symulacji."

1:15 do 2:00. Duży zestaw i konsola

Vladyslav uruchamia w przygotowanym PowerShell:

```
.\symulator.exe --konfiguracja konfiguracja.txt --zgloszenia przeciazenie.txt --wyniki
wyniki_pokazu --co 100
```

"Tutaj mamy sześć tysięcy transakcji i trzystu walidatorów. Co sto zakończeń konsola podaje statystyki od początku przebiegu. Terminowe i spóźnione dotyczą zakończonych zadań. Wykorzystanie obejmuje również pracę wykonaną w zadaniach, które jeszcze trwają. Program zapisuje pełne wyniki obu strategii."

Wskaż ostatni raport dla każdej strategii. Nie czytaj wszystkich 120 wierszy.

2:00 do 3:00. Co wynika z badań

Olesia pokazuje tabelę lub wykresy w `sprawozdanie.pdf`.

"Sprawdziliśmy małe obciążenie, przeciążenie i nagły napływ. Każdy zestaw uruchomiliśmy dla obu strategii i trzech ziaren, co daje osiemnaście przebiegów. Przy małym obciążeniu nie ma oczekiwania. Przy przeciążeniu FEE podnosi terminowość z około 13 do 17 procent. Przy nagłym napływie spada ona z 11,5 do około 6 procent. W naszych danych wysokie opłaty mają mniejszy zapas czasu. FEE daje im pierwszeństwo, ale nie pilnuje najbliższego terminu. Niskie opłaty czekają dłużej."

3:00 do 4:15. Kod i koszt działania

Vladyslav pokazuje `Symulator.cpp`, metody `uruchom` i `przydzielZadania`. Olesia wskazuje `Statystyki::oblicz`.

"Oba interfejsy używają tego samego silnika. Symulator dodaje nadejścia, przydziela pracę, przesuwa czas i zapisuje zakończenia. FIFO korzysta z queue, a FEE z priority_queue i naszego komparatora. Dane i wyniki przechowujemy w vector."

"Operacja kolejki FIFO ma koszt stały, a dodanie lub usunięcie w FEE kosztuje logarytmicznie względem liczby zadań. W każdym kroku sprawdzamy walidatorów, więc koszt zależy też od liczby kroków czasu. Pamięć rośnie wraz z liczbą transakcji i walidatorów. Dokładne wzory są w raporcie."

4:15 do 4:45. Powtarzalność i koniec

Olesia:

"FIFO i FEE dostają te same dane i listę losowań, ale osobne obiekty symulatora. Testy sprawdzają między innymi błędne dane, terminy, brak nakładania pracy i ponowne uruchomienie. Wszystkie osiemnaście wyników jest zachowanych. Do oddania przygotowaliśmy kod, dane, instrukcję, diagramy oraz sprawozdanie z tabelami i wykresami."

Zatrzymaj stoper. Jeśli próba przekracza pięć minut, pomiń demonstrację sortowania i skróć omówienie kodu. Zachowaj mały przykład oraz wnioski z trzech scenariuszy.

Gdy coś nie działa podczas pokazu

Jeśli GUI nie znajdzie danych, wskaż oba pliki przyciskami Wybierz. Jeśli nie można zapisać wyników, wybierz folder, do którego masz prawo zapisu, albo uruchom konsolę z innym `--wyniki`. Przed pokazem trzymaj projekt w zwykłym folderze użytkownika.

Jeśli nie ma kompilatora, uruchom dołączone programy Windows. Kompilację ćwiczymy wcześniej. Przy braku czasu pokaż zachowane wyniki w `eksperymenty/`; powiedz, że są to wyniki zapisanych przebiegów, a nie nowy przebieg.

Jak działa nasz program

Co symulujemy

Wyobraź sobie kolejkę zadań i grupę osób, które je wykonują. W naszym kontekście zadaniami są transakcje, a wykonawcami walidatorzy. Jedno zadanie może czekać, być wykonywane albo już się zakończyć. Walidator wykonuje jedną transakcję naraz.

Czas ma wartości 0, 1, 2 i kolejne liczby całkowite. Jeden krok jest umowną jednostką, a nie sekundą czekania przy komputerze. Program szybko oblicza kolejne kroki.

Model korzysta z pomysłu PoS, czyli Proof of Stake. Każdy walidator ma dodatnią liczbę stake, która określa wagę w losowaniu. Wolny walidator ze stake 30 ma trzy razy większą szansę od wolnego ze stake 10. Wszyscy wykonują tę samą transakcję tak samo długo. Nie dodajemy bloków, kryptografii ani uzgadniania stanu sieci.

Dane transakcji

Pole	Co oznacza
`id`	Numer służący do rozpoznania transakcji.
`nadejscie`	Krok, w którym transakcja trafia do kolejki.
`czas_wykonania`	Ile kroków zajmuje praca walidatora.
`fee`	Opłata. FEE używa jej jako priorytetu.
`deadline`	Krok, do którego transakcja powinna się zakończyć.

Wczytanie całego pliku na początku nie oznacza, że wszystkie zadania już nadeszły. Transakcję z nadejściem 10 wpuszczamy do kolejki dopiero w kroku 10.

W dużych danych czasy wykonania mają wartości od 8 do 32 kroków. Jest ich 25, więc powtarzają się. Deadline to nadejście plus czas wykonania i dodatkowy zapas. Zapas ma od 10 do 59 kroków i jest mniejszy dla większego fee. To założenie danych testowych, które trzeba pamiętać przy interpretowaniu wyników.

Dwie oddzielne decyzje

Pierwsza decyzja dotyczy kolejności transakcji. FIFO wybiera wcześniejsze nadejście, a przy remisie mniejsze ID. FEE wybiera większą opłatę, potem wcześniejsze nadejście i mniejsze ID. Deadline nie decyduje o kolejności żadnej z tych strategii.

Druga decyzja dotyczy wykonawcy. Losujemy tylko spośród wolnych walidatorów. Zajęci nie uczestniczą w losowaniu. Prawdopodobieństwo danego wykonawcy to jego stake podzielony przez sumę stake wolnych wykonawców.

W kodzie najpierw znajdujemy wykonawcę, a dopiero potem zdejmujemy zadanie z kolejki. Jeśli wszyscy są zajęci, zadanie zostaje w kolejce i nie zużywamy kolejnej wartości losowania.

Jeden przebieg krok po kroku

1. Ustawiamy czas 0, pustą kolejkę i wolnych wykonawców.

2. Dodajemy transakcje, których nadejście jest równe bieżącemu czasowi.

3. Przydzielamy zadania, dopóki są transakcje w kolejce i wolni walidatorzy.

4. Sprawdzamy, czy zostały przyszłe nadejścia, kolejka lub praca w toku. Jeśli nic nie zostało, kończymy.

5. Zwiększamy czas o jeden.

6. Zapisujemy zadania kończące się w tym kroku i zwalniamy wykonawców. Aktualizujemy liczniki, a co 100 zakończeń publikujemy raport.

7. Wracamy do dodawania nadejść.

Transakcja rozpoczęta w czasie 0 i trwająca 3 kroki kończy się w czasie 3. Ten sam walidator może dostać nową pracę w kroku 3. Rozpoczętego zadania nie przerywamy, nawet jeśli nadejdzie droższe.

FIFO i FEE liczymy osobno. Każdy wariant dostaje nowy obiekt `Symulator`, te same dane oraz tę samą listę losowań. Stan pierwszego wariantu nie przechodzi do drugiego.

Co mierzymy

Oczekiwanie to `start - nadejscie`. Dla nadejścia 3 i startu 8 wynosi 5. Czas wykonania liczymy osobno: `koniec - start`.

Terminowość sprawdzamy po zakończeniu zadania. Jeśli `koniec <= deadline`, transakcja jest terminowa. W przeciwnym razie jest spóźniona. Opóźnienie to większa z liczb: zero i `koniec - deadline`.

Wykorzystanie wykonawcy to procent czasu, w którym pracował od chwili 0. Gdy do kroku 10 przepracował 6 kroków, wykorzystanie wynosi 60%. Raport w trakcie uwzględnia też wykonaną część aktualnego zadania.

Jak czytać raport co 100 zakończeń

Pole konsoli	Znaczenie
`obsluzone`	Liczba zadań zakończonych od początku tego wariantu.
`czas`	Bieżący krok symulacji.
`srednie_oczekiwanie`	Średnia oczekiwania zakończonych zadań.
`maksymalne_oczekiwanie`	Największe oczekiwanie zakończonego zadania.
`terminowe`	Liczba zakończonych najpóźniej w deadline.
`spoznione`	Liczba zakończonych po deadline.
`wykorzystanie_srednie`	Średnia zajętości wszystkich wykonawców.
`min`, `max`	Najmniejsze i największe wykorzystanie pojedynczego wykonawcy.

Raport przy 200 obejmuje wszystkie pierwsze 200 zakończeń. Zadania czekające lub w toku nie są jeszcze liczone jako terminowe ani spóźnione. Zawsze `terminowe + spoznione = obsluzone`.

Tabela GUI

Domyślna kolejność to czas zakończenia, a przy remisie ID walidatora. Przy wielu wykonawcach zadanie rozpoczęte później może skończyć się wcześniej. Dlatego ID, nadejście i start nie muszą rosnąć w kolejnych wierszach.

Kliknięcie nagłówka porządkuje tylko indeksy widoku. Można wyświetlić transakcje według ID, startu lub innej kolumny. Drugie kliknięcie odwraca kierunek. Przycisk przywraca kolejność zakończenia. Pliki nadal zapisujemy w kolejności zakończenia.

Pliki programu: co robią

Pliki	Po co są
Dane.hpp	Wspólne dane transakcji, walidatorów i wyników.
Walidator.hpp, Walidator.cpp	Stan wykonawcy, początek i koniec jego pracy.
Symulator.hpp, Symulator.cpp	Czas, kolejki i przydzielanie transakcji.
Pliki.hpp, Pliki.cpp	Czytanie i sprawdzanie wejścia oraz zapis wyników.
Statystyki.hpp, Statystyki.cpp	Średnie, maksimum, terminy i wykorzystanie.
Przebiegi.hpp, Przebiegi.cpp	Uruchomienie FIFO i FEE na tych samych danych.
main.cpp	Konsola i wypisywanie raportów.
Gui.cpp	Okno, tabela i praca w osobnym wątku.
WidokWynikow.hpp	Sortowanie widoku tabeli.
Eksperymenty.cpp	Dane testowe i 18 przebiegów badań.
gui.rc, gui.manifest	Wygląd i ustawienia okna Windows.
build.cmd, testy.cmd	Kompilacja programu i testy C++.

Pytania, które warto przećwiczyć

1. Jaki problem rozwiązujecie?

Porównujemy dwa sposoby obsługi kolejki transakcji przy ograniczonej liczbie wykonawców. Sprawdzamy, jak kolejność wpływa na oczekiwanie, terminy i wykorzystanie walidatorów.

2. Co oznacza czas dyskretny?

Czas przyjmuje kolejne wartości całkowite. Sprawdzamy stan w krokach 0, 1, 2 i następnych. Jeden krok jest umowną jednostką. Program nie czeka sekundy po każdym zwiększeniu licznika.

3. Co z PoS jest w tym modelu?

Losowanie wolnego walidatora z wagą równą stake. Model obsługi kolejki nie obejmuje bloków, podpisów kryptograficznych, sieci ani uzgadniania stanu blockchaina.

4. Czy większy stake oznacza szybszego walidatora?

Nie. Wszyscy mają taką samą szybkość. Większy stake zwiększa szansę wyboru, kiedy wykonawca jest wolny. Gdy jest zajęty, w ogóle nie bierze udziału w losowaniu.

5. Czy FIFO zawsze kończy transakcje według ID?

FIFO wybiera do rozpoczęcia wcześniejsze nadejście, a przy remisie mniejsze ID. Przy kilku walidatorach i różnych czasach zadania kończą się w innej kolejności. ID samo w sobie nie określa nadejścia.

6. Jak rozstrzygacie remisy w FEE?

Najpierw fee malejąco. Przy tej samej opłacie wybieramy wcześniejsze nadejście, potem mniejsze ID. Taką regułę zapisuje `PorownajFee::operator()` w `Symulator.cpp`.

7. Dlaczego komparator kolejki zwraca true dla niższego priorytetu?

`priority_queue` używa komparatora do ustalenia, który element ma ustąpić innemu. Dla mniejszej opłaty zwracamy true, aby większa opłata była na szczycie. Dla równych opłat większy czas nadejścia i większe ID mają ustąpić mniejszym wartościom.

8. Do czego służą kontenery STL?

`vector` to lista elementów przechowywanych razem w pamięci. Używamy jej dla wejścia, wyników, walidatorów i losowań. `queue` pozwala dodawać na końcu i pobierać od początku. `priority_queue` udostępnia element o najwyższym priorytecie. To gotowe struktury standardowej biblioteki C++.

9. Czy kod wykorzystuje programowanie obiektowe?

Tak. `Walidator` przechowuje prywatny stan pracy i zmienia go przez metody. `Symulator` posiada wykonawców i zarządza przebiegiem. `Pliki` i `Statystyki` grupują operacje na wejściu, wyjściu i pomiarach. Nie używamy dziedziczenia, bo nie wymaga go ten model.

10. Dlaczego są pliki hpp i cpp?

Nagłówek określa dostępne typy i metody, a cpp zawiera ich działanie. Inne części programu dołączają nagłówek, żeby wiedzieć, co mogą wywołać. Podczas kompilacji trzeba dołączyć wszystkie potrzebne cpp. Samo dołączenie nagłówka nie dołącza implementacji funkcji.

11. Jak działa losowanie według stake?

Sumujemy stake wolnych walidatorów. Los z przedziału od 0 włącznie do 1 wyłącznie mnożymy przez tę sumę. Przechodzimy po przedziałach długości stake. Wykonawca, w którego przedział trafiliśmy, dostaje zadanie. Dla stake 10 i 30 pierwszy ma 25%, drugi 75% szansy.

12. Co to jest ziarno losowania?

Liczba ustawiająca początek generatora pseudolosowego. W tym samym programie to samo ziarno pozwala odtworzyć listę losowań. Używamy osobnego ziarna 20261102 do danych, a 11, 22 i 33 do wyboru wykonawców w badaniach. Inne implementacje biblioteki standardowej mogą inaczej przekształcać losowania na liczby ułamkowe.

13. Dlaczego oba warianty dostają te same losowania?

Chcemy porównać strategie na wspólnych warunkach wejścia. Każdy wariant zaczyna od pierwszego losu i ma świeży stan. Ten sam numer losowania oznacza kolejny przydział, a nie konkretną transakcję. Po zmianie kolejności FEE może przypisać ten los innemu ID.

14. Dlaczego zmiana ziarna nie zmienia średniego oczekiwania w wynikach?

Wykonawcy mają jednakową szybkość, więc wybór innego wolnego ID nie zmienia długości pracy. Dla tej samej strategii pozostaje ten sam harmonogram transakcji. Zmieniają się przydziały ID i wykorzystanie pojedynczych walidatorów.

15. Czy deadline określa priorytet?

Nie w naszych dwóch strategiach. Deadline służy do oceny po zakończeniu pracy. FEE kieruje się opłatą. Strategia wybierająca najbliższy deadline byłaby osobnym algorytmem, którego tu nie implementujemy.

16. Czy spóźnioną transakcję odrzucacie?

Nie. Wykonujemy ją do końca, zapisujemy spóźnienie i zwiększamy licznik spóźnionych. Deadline może być za krótki do realizacji, ale jeśli nie jest wcześniejszy niż nadejście, transakcja jest poprawnym wejściem.

17. Czym różni się oczekiwanie od opóźnienia?

Oczekiwanie to czas w kolejce przed rozpoczęciem pracy. Opóźnienie to przekroczenie deadline po zakończeniu. Zadanie może długo czekać i nadal zdążyć albo zacząć od razu, ale skończyć po zbyt krótkim terminie.

18. Jak liczycie wykorzystanie?

Dla każdego walidatora sumujemy czas pracy i dzielimy przez czas od 0 do chwili pomiaru. Wynik mnożymy przez 100%. Średnia obejmuje wszystkich walidatorów, także tych bez pracy. W raporcie w trakcie dodajemy wykonaną część zadań w toku.

19. Dlaczego przy raporcie 100 wykorzystanie może być bliskie 100%?

Sto zadań już się skończyło, ale inni wykonawcy mogą nadal pracować. Wykorzystanie mierzy zajętość zasobów. Nie jest procentem ukończenia wszystkich transakcji.

20. Po co trzy scenariusze?

Małe obciążenie sprawdza system z zapasem zasobów. Przeciążenie sprawdza rosnącą kolejkę przy zbyt szybkim napływie. Nagły napływ sprawdza jednorazową dużą kolejkę. W każdym przypadku mamy 6000 transakcji i 300 wykonawców.

21. Dlaczego czasy wykonania są różne?

Przyjęliśmy zadania o różnej długości. Generator wybiera od 8 do 32 kroków. Te same długości i opłaty zachowujemy pomiędzy scenariuszami. Zmieniamy nadejścia i przesuwamy deadline, aby zachować ten sam zapas.

22. Czy FEE jest lepsze od FIFO?

To zależy od miary i danych. W przeciążeniu FEE podnosi terminowość z 13,12% do 16,90%, lecz zwiększa maksimum oczekiwania z 333 do 384. Przy nagłym napływie terminowość spada z 11,50% do 6,02%. Wysokie opłaty czekają krócej, a niskie dłużej.

23. Co to jest głodzenie?

To sytuacja, w której zadanie stale przegrywa z nowymi zadaniami o wyższym priorytecie. Przy ciągłym napływie wysokich opłat niska opłata w FEE mogłaby czekać bez końca. Nasze pliki są skończone, więc wszystkie transakcje kończą pracę. W badaniach widzimy długie oczekiwanie, nie nieskończone głodzenie.

24. Co oznacza złożoność O()?

Opisuje, jak rośnie koszt wraz z rozmiarem danych. O(1) oznacza koszt niezależny od liczby elementów. O(log N) rośnie wolniej niż N. O(NK) oznacza przejście po N elementach dla każdego z K wykonawców albo odwrotnie. To oszacowanie liczby operacji, nie konkretny czas w sekundach.

25. Jaki jest koszt całego silnika?

N to liczba transakcji, K walidatorów, T liczba zwiększeń czasu. Sortowanie kosztuje O(N log(N+1) + K log(K+1)). Skany czasu to O((T+1)K), przydziały i końcowe wykorzystanie O(NK). Raporty kosztują O(RK), gdzie R to ich liczba. Dla dodatniego odstępu raportów R jest O(N+1). Pamięć wynosi O(N+K).

26. Dlaczego długie przerwy w nadejściach są niekorzystne?

Pętla przesuwa czas o jeden i nadal sprawdza walidatorów. Gdy kolejna transakcja nadejdzie bardzo późno, T rośnie mimo niewielkiej liczby zadań. W przyszłości można przeskakiwać do najbliższego zdarzenia, ale obecny program zachowuje pętlę krokową z planu.

27. Jak sprawdzacie poprawność?

Ręczny przykład ma znane starty, końce i średnią. Testy obejmują kolejność, remisy, zajętych walidatorów, błędne dane, puste wejście, terminy i raporty. Dla 18 przebiegów testy ponownie liczą miary ze 108000 zapisanych rekordów i sprawdzają, czy prace jednego wykonawcy nie nakładają się. Osobno sprawdzamy okno i sortowanie.

28. Czy GUI i konsola mogą liczyć inaczej?

Używają tych samych funkcji `wykonajPrzebiegi` i tych samych klas silnika. Różnią się obsługą użytkownika i domyślnym folderem zapisu. GUI zapisuje do `wyniki` obok wybranego pliku transakcji. Konsola domyślnie do bieżącego folderu lub do wskazanego przez `--wyniki`.

29. Po co obliczenia GUI są w osobnym wątku?

Wątek okna może wtedy nadal obsługiwać system Windows i odświeżać widok. Drugi wątek liczy przebiegi. Dostęp do przekazywanego wyniku chronimy mutexem, czyli blokadą przed równoczesną zmianą tych samych danych. Zamknięcie okna podczas pracy czeka na ukończenie obliczeń i zapisu.

30. Czy sortowanie tabeli zmienia FIFO lub FEE?

Nie. Sortujemy osobną listę numerów wierszy, według której GUI odczytuje oryginalne wyniki. Wiersze w plikach pozostają w kolejności zakończenia. Przy równych wartościach stabilne sortowanie zachowuje ich pierwotną kolejność.

Ćwiczenia przed prezentacją

Najpierw policz wynik, potem przeczytaj rozwiązanie. Na końcu uruchom mały zestaw w programie.

Ćwiczenie 1. FIFO i FEE

Jeden walidator. Wszystkie transakcje przychodzą w 0, trwają 2 kroki i mają deadline 10. T1 ma fee 1, T2 fee 9, T3 fee 4. Ustal kolejność, starty, końce i średnie oczekiwanie.

Rozwiązanie:

Strategia	Kolejność	Starty w tej kolejności	Końce	Oczekiwania
FIFO	T1, T2, T3	0, 2, 4	2, 4, 6	0, 2, 4
FEE	T2, T3, T1	0, 2, 4	2, 4, 6	0, 2, 4

Średnia to (0+2+4)/3 = 2. Maksimum to 4. Wszystkie transakcje są terminowe, koniec przebiegu to 6, a wykorzystanie jedynego walidatora wynosi 100%.

Sprawdzenie w konsoli:

```
.\symulator.exe --konfiguracja przyklady/konfiguracja.txt --zgloszenia przyklady/
zgloszenia.txt --wyniki wyniki_male --co 1
```

Ćwiczenie 2. Oczekiwanie i opóźnienie

Transakcja przychodzi w 3, startuje w 8, trwa 4 kroki i ma deadline 10. Ile wynoszą oczekiwanie, koniec i opóźnienie?

Rozwiązanie: oczekiwanie 8-3 = 5. Koniec 8+4 = 12. Opóźnienie 12-10 = 2. Transakcja jest spóźniona. Gdyby deadline wynosił 12, byłaby terminowa.

Ćwiczenie 3. Wykorzystanie

Mamy dwóch walidatorów. Do kroku 10 pierwszy przepracował 6 kroków, a drugi 4. Oblicz średnie, minimalne i maksymalne wykorzystanie.

Rozwiązanie: pierwszy 60%, drugi 40%. Średnia 50%, minimum 40%, maksimum 60%. Nie dzielimy czasu pracy przez liczbę zakończonych zadań.

Ćwiczenie 4. Losowanie stake

Dwaj wolni wykonawcy mają stake 10 i 30. Los wynosi 0,2. Kto dostanie zadanie? Co przy losie 0,5? Co, jeśli pierwszy walidator jest zajęty?

Rozwiązanie: suma wynosi 40. Pierwszy przedział to [0,10), drugi [10,40). Dla 0,2 pozycja to 8, więc wybieramy pierwszego. Dla 0,5 pozycja to 20, więc drugiego. Przy zajętym pierwszym jedynym kandydatem jest drugi, niezależnie od wartości poprawnego losu.

Ćwiczenie 5. Kolejność w tabeli

Dwaj walidatorzy rozpoczynają pracę w 0. T1 trwa 10 kroków, T2 trwa 2. Który wynik pojawi się pierwszy? Czy FIFO zostało naruszone?

Rozwiązanie: pierwszy będzie wynik T2 z końcem 2, potem T1 z końcem 10. FIFO może rozpocząć T1 przed T2 w tym samym kroku. Kolejność zakończeń wynika z długości pracy i równoległego wykonania.

Ćwiczenie 6. Remis w FEE

Trzy oczekujące zadania mają fee 9. T4 nadeszło w 1, a T2 i T3 w 0. Jaka będzie kolejność pobierania?

Rozwiązanie: T2, T3, T4. Wcześniejsze nadejście wygrywa przed porównaniem ID.

Ćwiczenie 7. Raporty narastające

Raport przy 100 zakończeniach ma 90 terminowych i 10 spóźnionych. W następnej setce kończy się 70 terminowych i 30 spóźnionych. Co pokaże raport przy 200?

Rozwiązanie: `obsluzone=200`, `terminowe=160`, `spoznione=40`. Raport obejmuje cały dotychczasowy przebieg. Z tych danych nie da się policzyć średniego oczekiwania ani wykorzystania, bo nie znamy czasów.

Ćwiczenie 8. Złożoność

Czy dwie transakcje zawsze oznaczają krótki przebieg? Jedna nadchodzi w 0, druga w kroku miliard.

Rozwiązanie: w obecnej pętli nie. Program przechodzi przez wszystkie kroki do drugiego nadejścia. Liczba transakcji N jest mała, ale liczba kroków T bardzo duża. Ten przykład wyjaśnia składnik zależny od T. Nie uruchamiaj go podczas prezentacji.

Próba przed obroną

Sprawdzenie przez każdą osobę

[ ] Uruchamiam GUI i konsolę z rozpakowanej paczki.

[ ] Potrafię wskazać poprawne pliki i folder zapisanych wyników.

[ ] Przed uruchomieniem przewiduję wynik małego przykładu.

[ ] Wyjaśniam różnicę między wyborem transakcji a wyborem walidatora.

[ ] Na przykładzie liczę oczekiwanie, opóźnienie i wykorzystanie.

[ ] Potrafię przeczytać jeden raport konsoli i jeden wiersz GUI.

[ ] W kodzie znajduję komparator FEE, pętlę czasu i obliczanie statystyk.

[ ] Wyjaśniam N, K i T oraz źródło kosztów w pętlach.

[ ] Podaję przykład, w którym FEE daje gorszy wynik niż FIFO.

[ ] Potrafię powiedzieć, co w naszym modelu oznacza PoS i czego model nie obejmuje.

Próba z drugą osobą

Pierwsza osoba prowadzi pokaz według planu z początku tego dokumentu, druga mierzy czas i zapisuje niejasne miejsca. Potem zamieńcie się rolami. Sprawdźcie też pytania o część kodu zwykle omawianą przez drugą osobę.

W dniu prezentacji

Skopiuj paczkę na drugi nośnik lub do drugiego folderu. Sprawdź program na komputerze, na którym odbędzie się pokaz. Jeśli to Windows bez kompilatora, użyj dołączonych plików exe. Na innym systemie potrzebna jest kompilacja konsoli; GUI jest dla Windows.

Przed pokazem otwórz okna i wybierz mały zestaw. Nie uruchamiaj generatora eksperymentów, jeśli nie chcesz odtworzyć danych i nadpisać jego wyników. Zachowane CSV i wykresy wystarczą do omówienia badań.

Po sprawdzeniu materiałów autorzy przekazują akceptację zmian. Dopiero potem można wykonać uzgodniony commit i zakończyć pracę nad projektem.